Guion sencillo para tu video

Qué mostrar, qué decir y qué hacer después.

Duración prevista: 8 minutos y 30 segundos. Ensaya y procura terminar antes de 9 minutos para tener margen. El máximo permitido es 10 minutos.

Antes de empezar

- Abre el proyecto en tu editor y deja lista la terminal en la carpeta raíz.

- Prepara README, main.py, solicitud.py, nodo.py, cola.py, solicitud_repository.py, soporte_service.py, json_solicitud_repository.py y gui.py.

- Prueba la aplicación con python main.py. Prepara una cola vacía; si existen datos, conserva una copia del JSON antes de preparar la demostración.

- En el grabador selecciona la pantalla y activa el micrófono. Graba 20 segundos para comprobar que se oye tu voz y se lee el código.

- Explica con tus propias palabras. No necesitas diapositivas. La parte central es mostrar y explicar el código, además de ejecutar el programa.

Cómo seguir la guía

EN PANTALLA indica qué abrir. DI contiene una explicación breve para practicar. SIGUIENTE indica dónde continuar. Señala con el cursor el código que estás explicando.

Presentación y estructura

1. Presenta el proyecto | 40 segundos

EN PANTALLA: README.md en modo Preview.

DI:

> Mi proyecto es un sistema de soporte técnico desarrollado en Python. Su objetivo es registrar solicitudes de clientes y atenderlas en orden de llegada. Continué el proyecto del primer parcial agregando persistencia en JSON y una interfaz gráfica con Tkinter.

SIGUIENTE: Muestra las carpetas del proyecto.

2. Explica la organización | 40 segundos

EN PANTALLA: La carpeta src desplegada, con sus subcarpetas.

DI:

> Organicé el código por responsabilidades. En models está la solicitud; en structures, la cola y los nodos; en repositories, el almacenamiento; en services, las validaciones; y en ui, la interfaz gráfica.

SIGUIENTE: Abre main.py.

3. Explica cómo empieza | 30 segundos

EN PANTALLA: La función main(). Señala las tres llamadas.

DI:

> Aquí empieza el programa. Primero creo el repositorio que utiliza el archivo JSON. Después creo el servicio y le entrego ese repositorio. Finalmente, inicio la interfaz gráfica.

SIGUIENTE: Abre src/models/solicitud.py y después src/structures/nodo.py.

Datos y lógica del proyecto

4. Explica los datos y la cola | 1 minuto y 20 segundos

EN PANTALLA: El constructor de Solicitud; después el de Nodo; finalmente enqueue, dequeue y peek en src/structures/cola.py.

DI:

> Cada solicitud tiene un identificador, el nombre del cliente y la descripción del problema. Cada nodo contiene un dato y una referencia al siguiente nodo. Con estos nodos construyo la cola.
>
> La cola funciona como una fila de atención. Enqueue agrega una solicitud al final y dequeue retira la del frente. Por eso se respeta el orden de llegada. Peek permite consultar la siguiente solicitud sin retirarla. También mantengo un contador para saber cuántas solicitudes hay.

SIGUIENTE: Abre src/repositories/solicitud_repository.py y luego src/services/soporte_service.py.

5. Explica el servicio | 1 minuto

EN PANTALLA: Los métodos del repositorio abstracto, brevemente. Luego registrar_solicitud y atender_solicitud del servicio.

DI:

> Este contrato define las operaciones que debe tener un repositorio. Así puedo cambiar el almacenamiento sin cambiar la lógica del servicio.
>
> El servicio verifica que el nombre y la descripción no estén vacíos. Luego genera un identificador, crea la solicitud y la guarda utilizando el repositorio. Para atender a un cliente, pide al repositorio la siguiente solicitud.

SIGUIENTE: Abre src/repositories/json_solicitud_repository.py.

Persistencia e interfaz gráfica

6. Explica la persistencia | 1 minuto

EN PANTALLA: Primero _cargar_datos y después _guardar_datos. Señala las llamadas a _guardar_datos dentro de guardar y obtener_siguiente.

DI:

> Al iniciar, este método lee el archivo JSON, reconstruye las solicitudes y las coloca en la cola.
>
> Este otro método recorre las solicitudes pendientes y las escribe en el archivo JSON. Se ejecuta después de registrar y después de atender. Por eso, al cerrar y abrir el programa, se recuperan los pendientes.

SIGUIENTE: Abre src/ui/gui.py.

7. Explica la interfaz | 1 minuto

EN PANTALLA: crear_widgets. Después registrar, atender y actualizar_estado.

DI:

> La interfaz está hecha con Tkinter. Aquí construyo los campos, las etiquetas y los botones. Cada botón utiliza command para llamar a su función.
>
> Registrar lee los campos y llama al servicio. Atender procesa al siguiente cliente. Actualizar estado muestra la cantidad de pendientes y la próxima solicitud. Los mensajes informan si una operación fue correcta o si faltan datos.

SIGUIENTE: Abre la terminal, ejecuta python main.py y muestra la ventana.

Demostración y cierre

8. Demuestra el funcionamiento | 2 minutos

EN PANTALLA: La aplicación. Comienza con la cola vacía para seguir este ejemplo.

1. HAZ: Pulsa registrar con los campos vacíos.
DI: El programa valida los campos y evita registrar una solicitud vacía.

2. HAZ: Registra Ana con el problema: Sin conexión a internet. Luego registra Luis con el problema: Impresora no imprime.
DI: Ahora hay dos solicitudes pendientes. Ana aparece primero porque llegó antes.

3. HAZ: Abre datos_solicitudes.json en el editor.
DI: Aquí podemos ver que las solicitudes se guardaron con su identificador, nombre y descripción.

4. HAZ: Regresa a la aplicación y atiende a Ana.
DI: Se atiende primero a Ana y Luis queda como siguiente. Esto demuestra el comportamiento FIFO.

5. HAZ: Cierra la ventana y ejecuta otra vez python main.py desde la misma carpeta.
DI: Luis continúa pendiente después de reiniciar. Esto demuestra que los datos se recuperan desde el archivo.

6. HAZ: Atiende a Luis y pulsa atender una vez más.
DI: La cola queda vacía y el programa muestra un aviso porque no hay más solicitudes.

9. Cierra la presentación | 20 segundos

EN PANTALLA: Deja la aplicación en pantalla.

DI:

> Este proyecto aplica una cola enlazada, separación de responsabilidades, persistencia en JSON e interfaz gráfica. Mostré cómo está organizado el código, cómo se registran y atienden las solicitudes y cómo se recuperan los datos después de cerrar el programa. Gracias.

SIGUIENTE: Detén la grabación y guarda el video.

Revisión y entrega

Antes de compartir el video

- Reproduce el video completo y comprueba que se escucha tu propia voz.

- Confirma que el código y la aplicación se ven y se leen.

- Comprueba que explicaste el objetivo, la estructura, las clases y métodos, la persistencia y la GUI.

- Verifica que se vea la ejecución, el orden FIFO y la recuperación de Luis al reiniciar.

- Confirma que la duración no supera los 10 minutos.

Publicación y enlaces

- Sube el video a YouTube como no listado, o a un servicio con acceso por enlace sin solicitar permisos.

- Deja el código fuente y README actualizado en la rama main de un repositorio público de GitHub o GitLab.

- El README debe contener nombre, descripción, objetivo, funcionalidades, persistencia, GUI, instrucciones de ejecución y lenguaje Python.

- Abre ambos enlaces en una ventana sin sesión iniciada para comprobar que el docente puede acceder.

- Entrega dos enlaces: el del video y el del repositorio.

Si el docente pregunta

¿Qué se guarda? Las solicitudes pendientes; no se guarda un historial de atendidas.

¿Cómo continúa el ID? Desde el mayor ID pendiente que se recupera. Si no quedan pendientes al reiniciar, la numeración empieza de nuevo.

¿Por qué usar una cola? Porque permite atender a los clientes en el orden en que llegaron.

¿Por qué separar el servicio del repositorio? Para separar las validaciones y operaciones del modo de guardar los datos.